Commands and lifecycle

Siddharth M. Bhatia · siddharth@sidmb.com
August 8, 2026

Contents

Launch and attach 1
Durable tasks 2
Inspect sessions 2
Synchronize and compare 3
Continuous synchronization 3
Forward local ports 4
Manage configured targets 5
Stop, remove, and uninstall 5

Launch and attach

```
fwd                              # human reuse workflow: attach or
     create interactively
fwd up                           # launch the configured default;
     stay local
fwd up runpod codex              # positional target and coding agent
fwd up --target work --agent codex
fwd up --new codex               # create a second session for
     this project
fwd up -- python train.py --epochs 10
fwd attach SESSION               # human terminal only
```

`fwd up` is idempotent and also repairs partially prepared sessions. Use `--new` only when you want a separate remote resource. Agent launches attach automatically only in a human terminal; `--detach` always stays local.

An explicit command runs as a durable task and returns its exit status. After two seconds, `Ctrl-C` cancels that task and `Ctrl-B` backgrounds the local viewer. `fwd attach` enters the primary tmux session; detach with `Ctrl-B D`.

Durable tasks

`fwd send` runs work in an existing session and never starts or restarts compute:

```
fwd send -- pytest -q
fwd send --name SESSION --detach -- python train.py
fwd send --ls --json
fwd send TASK_ID                # replay the log and continue following
fwd send TASK_ID --stop         # cancel only this task
fwd send cancel                 # cancel all queued tasks
fwd send cancel all             # cancel all active tasks
```

Use `--` before commands whose flags belong to the remote program. Invoke a shell explicitly for pipes, redirects, and globs:

```
fwd send -- bash -lc 'cat outputs/*.json | jq .'
```

On supported backends, `--stop-after` queues remote-owned shutdown after new work completes. `fwd send stopafter` queues shutdown after all current work, and `fwd send cancel stopafter` disarms it. Lambda does not support remote stop-after; use local `fwd stop` after retrieving results.

Inspect sessions

```
fwd ls
fwd ls --all-projects
fwd ls --json
fwd ls --columns backend,status,ports
fwd doctor --json
fwd info --json
```

Rich tables are used in terminals. Redirected or agent output defaults to Markdown; `--json` provides stable machine-readable output. Progress and diagnostics stay on stderr.

Commands accept exact session names and, where unambiguous, target or backend aliases. Use selectors positionally with `attach`, `stop`, `rm`, and `diff`; use `--name` with `send`, `push`, and `pull`. `stop` and `rm` accept multiple selectors.

Synchronize and compare

```
fwd diff                          # unified diff; no changes to either side
fwd diff -q                       # exit 0 same, 1 different, 2 error
fwd push                          # mirror local files to remote
fwd pull                          # additive whole-project download
fwd pull outputs/ logs/           # additive path-scoped download
```

Synchronization honors `.gitignore`, `.fwdignore`, and configured exclusions. Upload includes `.git/` so remote agents retain repository context; pull and diff exclude `.git/`, so pulling does not transfer remote commits. Export a patch or Git bundle when you need a remote commit locally.

Push mirrors the local synchronized tree and normally deletes remote-only synchronized files. Pull is always additive and never deletes local files. Uploads are capped by `sync.max_size_gb` (1 GB by default) and use a staging directory so an interrupted or over-limit upload does not replace the live remote project.

Continuous synchronization

`fwd sync` keeps the local project and the remote directory converged in both directions while a session runs, instead of waiting for an explicit `push` or `pull`. It is built on Mutagen, is **off by default**, and is stored per target:

```
fwd sync on                       # enable for this target and start it now
fwd sync status                   # configured intent, live state,
     and conflicts
fwd sync status --json
fwd sync off                      # stop syncing; the remote directory is
     left as-is
fwd sync on SESSION               # act on a named session instead of
     this directory's
```

Toggling takes effect immediately, including on a session that is already running. Enabling it also enables it for future launches on that target; `fwd up` and `fwd attach` start the sync automatically whenever it is on.

Key properties:

- `.git` **is never continuously synced.** Both sides run Git concurrently, and a half-propagated index or packfile corrupts the repository. `fwd push` and `fwd pull` still carry `.git`, and remain the way to move repository state.
- **Conflicts are never resolved destructively.** The mode is Mutagen's `two-way-safe`, so a file edited on both sides is reported by `fwd sync status` rather than overwritten. Settle one by deleting the copy you do not want on one side; the surviving version then propagates. Editing the losing side again does not help — that is simply another competing change.
- **The initial transfer still goes through rsync.** `fwd up` performs its usual size-bounded push, and Mutagen only moves steady-state deltas afterwards, so `sync.max_size_gb` still governs the large transfer.
- **Ignore rules are captured when the sync starts**, translated from `.gitignore` (including nested files), `.fwdignore`, and `sync.exclude`. After editing those, run `fwd sync off && fwd sync on` to pick up the change.
- **Not every transport supports it.** Mutagen installs a remote agent over `scp`, which the RunPod SSH proxy cannot run; fwd warns and skips continuous sync there.

fwd installs and drives Mutagen itself: the remote agent installs automatically on first connection, and the first local use offers to run `brew install mutagen-io/mutagen/mutagen` when Homebrew is available. fwd runs its own isolated Mutagen daemon under `~/.fwd/mutagen`, so it never disturbs a Mutagen daemon you run yourself.

Forward local ports

```
fwd ports 3000                 # local 3000 to remote 3000
fwd ports work 8080:3000       # local 8080 to remote 3000
fwd ports --ls
fwd ports --close 3000
fwd ports work --close         # close all forwards for one session
fwd up --ports 3000 --ports 8080:3000
```

Forwards bind local and remote loopback only. fwd refuses occupied local ports before changing tunnel state and tracks persistent OpenSSH control masters across CLI processes.

Manage configured targets

`fwd targets` manages saved targets — the `[targets.NAME]` configuration entries that describe *how to reach* compute. The session commands above manage the compute itself.

```
fwd targets ls                  # name, backend, key connection detail,
     default marker
fwd targets ls pod              # only targets whose name contains "pod"
fwd targets add ssh --host my-box --target-name work
fwd targets info work           # resolved values, which are defaults, and
     tracked sessions
fwd targets update work         # setup again, prefilled with the current
     values
fwd targets rm work             # remove the configuration entry only
```

`fwd setup` remains a permanent alias for `fwd targets add`. All of these read local configuration and session state only; no provider is contacted. `update` and `rm` open a picker when no target is named and fail with an actionable message when nothing can answer a prompt; `rm` requires `--force` non-interactively. Removing a target never stops or destroys compute — use `fwd rm` for that — and removing the default repoints `default_target` at the sole survivor or clears it.

Stop, remove, and uninstall

```
fwd stop SESSION                # stop compute; retain configured
     persistent storage
fwd rm SESSION                  # destroy remote resources and forget
     the session
fwd rm one two                  # operate on several resolved sessions
fwd rm --all                    # destroy every tracked session
     after confirmation
fwd uninstall                   # remove local fwd data, skill,
     and completion
```

`stop` and `rm` check the remote Git worktree and refuse when reachable work is dirty. Resolve that work with `fwd diff`, `fwd pull`, or a remote commit before retrying. Force flags explicitly accept possible data loss.

`fwd rm` is irreversible. `fwd uninstall` does not destroy remote resources and should normally follow `fwd rm --all`.

See Troubleshooting for recovery and safety details.

© 2026 Siddharth M. Bhatia. All rights reserved.